Senior Data Engineer Challenge #2

Implement an Entity Extraction Pipeline in Airflow

Overview

In this paid task, you will implement a small but functional version of an entity extraction and matching pipeline, which:

a. Demonstrates core functionality, and;

b. Is to be deployed on your local environment.

This task involves Airflow, Docker, and some database schema design.

It should take about 4 hours to complete.

Context

At Bilby, we collect publicly available documents, such as government documents or news, and then enrich them using various algorithmic methods, in order to create insights for downstream analysis. One such enrichment process is the ability to identify and extract *entities* — defined in the box below — from these documents, in a robust data pipeline.

> 💡 **Definition:** For Bilby, an ***entity*** is any concept or object that can be identified and tracked within our data system. Entities represent key elements within government policy documents and related sources.
>
> Conceptually, entities range from concrete objects like persons, organisations, and geopolitical entities to abstract concepts like tariffs, events, or economic threats. Each entity type has specific attributes that capture relevant information and relationships. Under this definition, there are infinitely many entity types. To help you fix ideas, here is a short list of examples:
>
> - `person`: Individual actors such as government officials, business leaders, and other notable figures.
> - `company`: Commercial organisations operating in various sectors.
> - `location`: A region on planet earth. Note that because Bilby is interested in government, each extracted location in this exercise will actually be a geopolitical entity ("GPE"). However, this distinction is irrelevant to this task.
>
> Each entity is stored as a structured object with attributes specific to its type, and may include relationships to other entities.

Bilby works with two kinds of entities: *Extracted entities*, and *Source of Truth (SoT) entities*:

1. ***Extracted entities*** are the mentions of entities detected directly in the documents we scrape. For example, a document may include the snippet: "...in speech a yesterday, President Xi Jinping announced new funding for batter manufacturing in Guangdong province...". Each of the two strings highlighted in pink represents an extracted entity.
2. In contrast, ***SoT entities*** are entities that Bilby manually curates and stores in an internal database. These entities represent politically or commercially important people, events or organisations, and their database records include all relevant metadata for our customers. Here is an example:

```
{
  "entity_name": "Chen Wenqing",
  "entity_attributes": {
    "type": "Decision Maker",
    "entity_aliases": [
```

```
  "Wenqing Chen",
  "Chen W."
 ],
 "date_of_birth": "1960-01-24",
 "currently_active_as_decision_maker": true,
 "actor_importance": 1,
 "institution": "Central Political and Legal Affairs Commission",
 "function": "Secretary",
 "party_affiliation": "Communist Party of China",
 "political_party_or_parliamentary_group_english": "Communist Party of China",
 "political_party_or_parliamentary_group_native": "中国共产党",
 "sector_taxonomy_key_words": [
  {
   "level_1": "Public Administration & Security",
   "level_2": "National Security",
   "level_3": "Intelligence"
  },
  {
   "level_1": "Public Administration & Security",
   "level_2": "Legal Affairs",
   "level_3": "Judicial Oversight"
  },
  {
   "level_1": "Public Administration & Security",
   "level_2": "Counterterrorism",
   "level_3": "Domestic Extremism"
  },
  {
   "level_1": "Public Administration & Security",
   "level_2": "Internal Stability",
   "level_3": "Social Governance"
  }
 ],
 "job_title_english": "Secretary of the Central Political and Legal Affairs Commission",
 "job_title_native": "中央政法委员会书记",
 "memberships_full_name_english": [
  "Politburo of the Chinese Communist Party",
  "Central Committee of the Chinese Communist Party"
 ],
 "memberships_full_name_native": [
  "中国共产党中央政治局",
  "中国共产党中央委员会"
 ],
 "level": "central",
 "highest_education": "Southwest University of Political Science & Law",
 "country_specific_attributes": {
  "China": {
   "ethnicity": "Han",
   "birthplace": "Renshou County, Sichuan",
   "party_member": true,
   "party_join_date": "1983-03-01",
   "weibo": "weibo.com/chenwenqing_official",
   "wechat": "ChenWenqing_CN"
  }
 }
```

```
  },
  "entity_type": "Person",
  "entity_country_origin": "China",
  "last_updated": "2025-01-15T14:30:45Z"
}
```

Matching extracted entities with their corresponding SoT records provides consumers of our data with useful extra context on the people, places and organisations mentioned this data.

Part of the DAG you will implement in this exercise models this matching procedure in a simple way, using the notion of an ***alias***. An ***alias*** is any term that refers to a SoT entity. For example, here is a list of possible aliases for Xi Jinping:

```
"Xi Jinping": [
      "President Xi",
      "Chairman Xi",
      "General Secretary Xi",
      "Xi",
      "Chinese President",
      "China's Leader",
      "Core Leader",
      "Mr. Xi",
      "Communist Party Leader",
      "China's Paramount Leader"
    ],
```

Note: For simplicity, this exercise will not deal directly with an SoT entities database. Instead, we include a hard-coded mapping between SoT entities and their aliases, to facilitate a simple matching algorithm.

Objective

The objective of this challenge is to build a functioning entity extraction and matching pipeline that demonstrates your ability to:

1. **Design and implement an Airflow DAG** that orchestrates the entire pipeline process.
2. **Create a Docker container** for the entity extraction component to isolate the model loading from the Airflow runtime (Note: This allows us to avoid the `SIGSEGV` errors that occur when one tries to load a model directly in Airflow).
3. **Design an efficient** data structure to store the total output of the pipeline.
4. **Implement the core functionality** of extracting entities from documents and matching them against a known aliases table.

Note: This task is designed for you to demonstrate knowledge of Airflow, by building a proof-of-concept pipeline that extracts and matches entities. Accordingly, "success" means that the pipeline runs end-to-end and incorporates best practices where possible. **However, the code does not need to be optimised for production.**

Getting started

Prerequisites

Before starting the task, please ensure that both the following are installed on your computer:

- The package manager uv, and;
- A container runtime like Docker.

Environment setup

To help you, we have provided you with:

- A git repo to help you set up Apache Airflow easily on your computer. Download it here.

  entity-extraction-pipeline-impl.zip

- A table of entities and its aliases (`data/entity_aliases.csv`)
- Sample documents, from which you will extract the entities (`data/documents.csv`)

Dev environment setup

> 💡 Before running the following command, please ensure that the `AIRFLOW_HOME` environment variable is not set (for example, by running `unset AIRFLOW_HOME` in the terminal).

After downloading the zip file containing the repo, please run the following command **in the project's root directory**, to create a `.env` file:

```
cp .env.example .env
echo "AIRFLOW_HOME=$(pwd)/" >> .env
```

Afterwards, run the following to start an airflow instance:

```
uv run --env-file .env airflow standalone
```

The Airflow webserver should be available at localhost:8080. The login credentials are visible in the terminal output, as well as in the file `standalone_admin_password.txt` in the project root directory.

Technical specs

At its core, the entity extraction pipeline can be depicted as a four-step chain:

```
graph TD
LoadDocuments[Load Documents]
--> EntityExtraction[Entity Extraction]
--> EntityMatching[Entity Matching]
--> StoreToDB[Store to Database]
```

Below, we outline each step in more detail. Note: The DAG you in your solution may be more complex than the one above, for example to accommodate error management — this is totally OK.

Load documents

This task reads the documents from a local CSV file, to simulate pulling data from upstream. The sample documents are provided in the `data/documents.csv` file in the repository. This file contains columns including `uuid` and `body_en` . This latter column contains the text to be processed.

Entity extraction

Actually building an entity extraction model is NOT part of this task. Instead, we will use a pre-trained, open source GLiNER model. We will apply this model, using code outlined below, to the document texts contained in the `body_en` field of the CSV file provided. The goal will be to extract the following three entity types: `["Person", "Company", "Location"]` .

Note: The GLiNER model is a large machine learning model that requires significant memory. Running it directly in Airflow tasks can cause memory errors. For this reason, we recommend isolating it in a Docker container that can be called from your Airflow DAG.

> 💡 The model will be downloaded automatically, the first time it is used. Consider implementing caching in your Docker container to avoid downloading the model on every run, which would improve performance for repeated executions.

Below, we have included example code for entity extraction, for your convenience. To use this code please first install the `gliner` package (for example, by running `uv add gliner` to add it to your project).

1. To load the model, you can use the following snippet:

```
from gliner import GLiNER

def load_gliner_model() -> GLiNER:
    """
    Load a pre-trained GLiNER model. If it hasn't been downloaded yet, it
    will be downloaded from the Hugging Face Model Hub automatically.

    Returns:
        An instance of the pre-trained GLiNER model.
    """
    logging.info("Loading GLiNER model...")
    return GLiNER.from_pretrained("urchade/gliner_multi-v2.1")
```

2. To run the model, you can use the following snippet:

```
def extract_entities(
    model: GLiNER,text: str, labels: list[str] | None = None, threshold: float = 0.5
) -> list[dict]:
    """
    Extract entities from a given text using a GLiNER model.

    Args:
        model: The GLiNER model to be used for entity extraction.
        text: The text from which to extract entities.
        labels (optional): The entity types to be extracted, in lower case or title case.
        threshold (optional): The minimum confidence score required for an entity to be returned.

    Returns:
        A list of dictionaries, each containing the text and start and end indices of an extracted entity.

    Example Usage:
    ```
        res = extract_entities(model, text, labels, threshold)
    ```

    Example Output:
    ```
        [
            {
                'start': 5,
                'end': 40,
```
```

```
            'text': 'Cristiano Ronaldo dos Santos Aveiro',
            'label': 'Person',
            'score': 0.9360320568084717
        },
        ...
    ]
```
"""
if labels == None:
    labels = ["Person", "Company", "Location"]

entities = model.predict_entities(text, labels, threshold=threshold)
return entities
```

## Entity matching

After extracting the entities, we need to match them against our entity aliases table for disambiguation. For example, suppose we have the following data:

`document`: "…. in January, Chairman Xi gave a speech about technological development…"

`extracted_entity_text`: "Chairman Xi"

`entity_aliases` for Xi Jinping:

```
{
  "name": "Xi Jinping",
  "aliases": ["President Xi", "Chairman Xi"]
}
```

Then the matching algorithm would observe that the extracted entity text "Chairman Xi" appears in the list `["President Xi", "Chairman Xi"]` (which is the value of the "aliases" key), and conclude that the SoT entity has official name "Xi Jinping". More generally, matching can be implemented as a two-step process:

1. Compare the extracted entity text against in our aliases table. This table contains the columns:
    - `entity_type`: `str`
    - `name`: `str`
    - `aliases`: `list[str]`
2. A successful match occurs when the extracted text exactly equals either:
    - The `name` field, OR;
    - Any entry in the `aliases` list.

Please note that:

- The `name` value is not included in the `aliases` list.
- Matching should be case-sensitive (exact matches only).
- If an entity doesn't match any entry in the `aliases` table, it should still be recorded in your output with a null/empty reference to the `aliases` table.

The `aliases` table is provided in the `data/entity_aliases.csv` file in the repository.

Store to database

The final task stores the extracted data in a structured format in a database. However, before you can store this data, you must first design a simple schema for it.
```

> 💡 For simulation purposes, we will save this structured data (as a CSV, JSON, Parquet, SQLite, or other — this is up to you) in the `output/` directory.

Your output schema should cleanly and clearly capture the three-way relationship between:

1. Extracted entities;
2. The documents from which they were extracted, and;
3. The SoT entities corresponding to the extracted entities.

Your schema should also explicitly capture the position of extracted entities within their parent documents. Below is an example output structure. It is included here for illustrative purposes only — you are free to submit any alternative satisfying the above requirements. The table has the following fields:

- `uuid` (document id)
- `[... and the rest of the existing fields in documents.csv]`
- `entities`, where entities is a list of:
  - `entity_type`
  - `entity_text`
  - `start_pos`
  - `end_pos`
  - `is_matched`
  - `matched_entity_id`
  - `matched_entity_name`

Functional requirements

Your implementation must meet the following functional requirements:

1. **DAG structure**: Create an Airflow DAG that logically separates the tasks into something that is easy to follow, like:
   - Load Documents
   - Entity Extraction
   - Entity Matching
   - Store to Database

   You may add additional tasks if needed for your implementation (such as spinning up docker containers, data validation, error handling, etc.), but the core flow should follow this logical progression.
2. **Docker integration**: Implement a container for the entity extraction process. For communication between Docker and Airflow, you may use any of the following solutions:
   - Volume mounts to share data
   - An API endpoint exposed by the Docker container
   - Direct command execution with input/output parameters

   Choose the method that best fits your implementation approach.
3. **Data processing**:
   - Successfully load and process the provided sample documents.
   - Extract entities of types: `Person`, `Company`, and `Location`.
   - Match extracted entities against the provided `aliases` table.

- Store the results in a structured schema.

4. **Error handling**: Implement appropriate error handling for each task in the pipeline.
   - Manage potential failures in the entity extraction process.
   - Log errors in a way that makes troubleshooting possible.
5. **Logging**: Include logging throughout the pipeline to track the extraction and matching process.

Implementation requirements

Your implementation should satisfy the following technical requirements:

1. **Code Quality**:
   - Write clean, readable code and follow best practices.
   - Include comments where necessary (e.g. explain why a certain design choice was made).
2. **Docker Configuration**:
   - Create a Dockerfile for the entity extraction component.
   - This container should be easy for the evaluator to build and deploy.
   - Document any environment variables or configuration needed.
3. **Output Schema**:
   - Design a schema that satisfies all the requirements mentioned above.
4. **Airflow DAG**:
   - Create a properly structured Airflow DAG.
   - Configure task dependencies correctly.
5. **Performance Considerations**:
   - No obvious performance bottlenecks.
   - Consider the scalability of your solution for larger datasets.
   - While there are no strict performance benchmarks, your solution should process the sample documents in a reasonable time (under a few minutes).

Your deliverables

Please provide the following deliverables:

1. **Source Code**:
   - Complete implementation of the Airflow DAG.
   - Docker configuration files.
   - Any additional scripts or utilities.
2. **Documentation**:
   - README file explaining how to set up and run your solution.
   - Brief documentation of your database schema design.
   - Any assumptions or design decisions you made.
3. **Output Data**:
   - Sample output CSV file demonstrating successful entity extraction and matching.
4. **Setup Instructions**:

   Your written instructions should:

- Be clearly written.
- Explicitly provide any configuration steps needed.
- Be short and as simple to follow as possible — do not write down 17 steps when only five are needed.

Our evaluation criteria

Your solution will be evaluated based on the following criteria:

1. **Functionality**: Does the solution meet all the functional requirements?
2. **Sensible naming:** Are the tasks' names well-defined? Can others tell what is happening in a task by just looking at the name?
3. **Code quality**: Is the code well-structured, documented, and maintainable?
4. **Output schema design**: Is the output schema well-designed, and does it meet all requirements?
5. **Docker implementation**: Is the Docker container properly configured?
6. **Error handling**: Does the solution handle errors gracefully?
7. **System reproducibility:** Can your submission be easily reproduced on other machines, such as the evaluator's?
8. **Creativity**: Bonus points will be awarded for any additional features or improvements beyond the basic requirements.

Submission protocol

Submit your code in a GitHub repository. All text documentation should be in markdown format and included in the repo. A complete solution will include at least the following:

1. All source code files.
2. Docker configuration files.
3. Documentation files.
4. Sample output files.

Once you have completed this task, Bilby will be in touch to get your bank details, so we can transfer you the USD 200.00 completion fee.

> 💡 We believe the above instructions are reasonably clear. However, there may remain some ambiguities. If so, it is your responsibility to resolve these yourself, without further input from us. One exception: Please contact us immediately if you are unable to access the code or data necessary to begin the task, and we will fix this promptly.

We wish you the best of luck in completing this task!